CITS3002 - Project - Mini Internet Protocol Stack Simulator

24147733 - 22708177
Evan Mioceich - Thomas Nylund

Overview

This project implements a simplified network stack simulator in Python, demonstrating how Layer 2 (Data Link), Layer 3 (Network), and Layer 4 (Transport) work together to deliver data between two hosts across a router. The simulation models a fixed topology: Host A → Router R1 → Host B, across two subnets (10.0.1.0/24 and 10.0.2.0/24). All communication is logical, no real networking libraries are used.

File Organisation

config.py defines all fixed network parameters including IP addresses, MAC addresses, ARP tables, routing tables, and protocol constants. All other files import from here, meaning any network parameter can be changed in one place. This centralisation avoids hardcoded values scattered across the codebase and makes the topology easy to reason about.

protocol.py contains two categories of classes. The first are header classes: UDPSegment, IPPacket, and EthernetFrame, which represent the data structures for Layers 4, 3, and 2 respectively. Each class stores its header fields as plain Python attributes and nests the layer above it as its payload, directly implementing the principle of encapsulation. A UDPSegment is the innermost structure, wrapped by an IPPacket, which is in turn wrapped by an EthernetFrame. This mirrors how real network stacks operate. The second category are layer behaviour classes: TransportLayer, NetworkLayer, and DataLinkLayer, which implement the logic each device performs at each layer when sending or receiving data. Separating data structures from behaviour was a deliberate design decision. It keeps protocol.py focused on what packets look like, while devices.py focuses on what devices do with them.

devices.py contains the Host and Router classes. Each device owns instances of its layer classes, passed a reference to the parent device on construction so that layers can communicate with each other via the device (for example, TransportLayer calling self.device.network.receive_from_above). This back-reference pattern avoids layers needing direct references to each other, keeping coupling low. Hosts have all three layers while routers have only Layer 2 and Layer 3, since routers do not process transport-layer data. Both classes provide an add_neighbour method which appends a device to the DataLinkLayer's neighbour list, allowing frames to be passed between devices in the simulation.

main.py is the entry point. It reads the message size from the command line, creates the three devices with their configurations from config.py, wires them together as neighbours, and initiates the transmission by calling hostA.transport.receive_from_above with the size and destination IP. The destination IP is provided here at the application level and passed down through the layers, reflecting how real applications specify a destination when initiating a connection.

Main Components

UDPSegment encapsulates application data with source and destination ports, a computed 16-bit checksum, a segment type (DATA or ACK), and a sequence number. The checksum is computed by summing all bytes of the payload modulo 65536, producing a value that fits in the 2-byte checksum field. This allows the receiver to detect corruption by recomputing the checksum and comparing it to the stored value. The sequence number alternates between 0 and 1 to support the rdt2.2 alternating bit protocol.

IPPacket wraps a UDPSegment with source and destination IP addresses, a TTL value starting at 100, and a protocol identifier set to 17 to indicate a UDP-like payload. The total length field is computed as 12 bytes of IP header plus the length of the encapsulated UDPSegment. The TTL is decremented at each router hop and the packet is dropped if it reaches zero, preventing packets from looping indefinitely.

EthernetFrame wraps an IPPacket with source and destination MAC addresses and an EtherType field set to 0x0800 to identify the payload as IPv4. Unlike IP addresses which remain constant end-to-end, MAC addresses change at every hop. When Router R1 forwards a packet, it re-frames it with its own outgoing interface MAC as the source and the next device's MAC as the destination.

TransportLayer handles segmentation of large messages (greater than 500 bytes) into multiple chunks, each sent as a separate UDPSegment. Segmentation uses slice notation to extract each 500-byte chunk from the data. The rdt2.2 mechanism works synchronously in this simulation: because Python executes the entire send and receive chain inside a single call stack, the ACK from the receiver arrives and is processed before the next segment is sent. The sequence number is only flipped when an ACK is received, not when a segment is sent, to avoid the double-flip bug that would cause all segments to use the same sequence number.

NetworkLayer handles IP packet creation and routing decisions using a routing table lookup based on bitwise prefix matching. Each destination IP and network address is converted to a 32-bit integer using bit shifting, and a bitmask derived from the prefix length is applied to both before comparison. This correctly handles routes of any prefix length, including the default route (0.0.0.0/0) which matches everything. When a packet arrives at a router, the TTL is decremented before forwarding. When a packet arrives at a host, the destination IP is compared against the device's own IP to determine local delivery. For the router, which has two interface IPs, this comparison is made against a list of all interface IPs.

DataLinkLayer handles Ethernet frame creation and delivery. When sending, it looks up the destination MAC from the device's ARP table using the next-hop IP provided by the network layer. For routers, the source MAC is taken from the outgoing interface's entry in the iface_mac dictionary rather than a single device MAC, since each router interface has its own MAC address. When receiving, the arriving interface is determined by finding which interface MAC of the router matches the frame's destination MAC. Hosts and routers are distinguished using hasattr checks for the iface_mac attribute, which only routers possess. MAC learning is implemented as a log output to satisfy the specification requirement, though in this fixed topology the ARP tables are pre-populated and the learned values do not influence forwarding decisions.

How to Run

python3 main.py <size>

Where <size> is the application message size in bytes. For example:

python3 main.py 10
python3 main.py 600

Messages larger than 500 bytes are automatically segmented into multiple transport-layer segments, each transmitted sequentially using the rdt2.2 protocol.